IIT Mandi Hackathon

Student AI Study Companion

A-to-Z Product, AI, Engineering and Hackathon Roadmap

Team Project Blueprint

October 3, 2026

Contents

Executive Summary 4

1 1. Problem Definition 5
- 1.1 The Student Problem . . . 5
- 1.2 Proposed Solution . . . 5
- 1.3 Success Criteria . . . 5

2 2. Product Scope 7
- 2.1 MVP . . . 7
- 2.2 Stretch Features . . . 7

3 3. End-to-End Architecture 9
- 3.1 Recommended Stack . . . 9

4 4. Architecture by Data Flow 10
- 4.1 Upload Flow . . . 10
- 4.2 Question Answering Flow . . . 10
- 4.3 Grounding Rule . . . 11

5 5. Retrieval-Augmented Generation (RAG) 12
- 5.1 Why RAG Instead of Training . . . 12
- 5.2 Chunking . . . 12
- 5.3 Retrieval Improvements . . . 13

6 6. Multimodal Ingestion 14
- 6.1 PDF . . . 14
- 6.2 PPTX . . . 14
- 6.3 Images and Diagrams . . . 14
- 6.4 Lecture Videos . . . 14

7 7. Database Design 15
- 7.1 Core Tables . . . 15
- 7.2 Mastery Model . . . 15

8 8. Backend API 16
- 8.1 Suggested Endpoints . . . 16
- 8.2 Backend Request Flow . . . 16

9 9. Frontend 17
- 9.1 Pages . . . 17
- 9.2 Ideal Student Journey . . . 17

10 10. AI Prompt Design **18**
10.1 System Prompt Responsibilities . . . 18
10.2 Quiz Generation Contract . . . 18

11 11. Hallucination and Safety Controls **19**
11.1 Failure Cases . . . 19
11.2 Prompt Injection Principle . . . 19

12 12. Evaluation **20**
12.1 Test Dataset . . . 20
12.2 Metrics . . . 20
12.3 Latency Budget . . . 21

13 13. Development Roadmap **22**
13.1 Phase 0 — Team Alignment . . . 22
13.2 Phase 1 — Repository . . . 22
13.3 Phase 2 — Backend Skeleton . . . 22
13.4 Phase 3 — Ingestion . . . 23
13.5 Phase 4 — RAG . . . 23
13.6 Phase 5 — Frontend . . . 23
13.7 Phase 6 — Assessment . . . 23
13.8 Phase 7 — Testing . . . 24
13.9 Phase 8 — Deployment . . . 24

14 14. Team Division **25**

15 15. Git Workflow **26**

16 16. Environment Variables **27**

17 17. Deployment Architecture **28**

18 18. Hackathon Demo Script **29**
18.1 Demo Narrative . . . 29

19 19. What Can Go Wrong **30**
19.1 API Key Failure . . . 30
19.2 CORS Failure . . . 30
19.3 Slow Upload . . . 30
19.4 Bad Answers . . . 30
19.5 Citations Look Correct but Are Wrong . . . 30
19.6 Demo Internet Failure . . . 30

20 20. Security and Privacy **31**

21 21. Product Differentiation **32**

22 22. What Not To Do **33**

23 23. Final Technical Checklist **34**

24 24. Final System Diagram **35**

One-Sentence Pitch **36**

Final Principle **37**

Executive Summary

The project is an AI study companion that turns a student's own learning material into a source-grounded tutor. The student can upload PDFs, PPTs, images/diagrams and lecture videos. The system extracts and indexes the material, retrieves relevant evidence, answers questions with citations, generates adaptive quizzes, tracks topic mastery, and uses the student's results to personalize later practice.

Core principle: do not train a giant foundation model from scratch. The hackathon MVP should use an existing multimodal LLM through an API and build the differentiated product around ingestion, retrieval, citations, assessment, personalization, evaluation, and a polished student workflow.

Chapter 1

1. Problem Definition

1.1 The Student Problem

Students have information scattered across lecture PDFs, PPTs, handwritten notes, screenshots, diagrams, and recordings. Generic chatbots can answer questions, but they may use information outside the student's syllabus, hallucinate, fail to cite the exact lecture source, and give the same explanation to every student.

The project therefore targets five concrete problems:

1. **Fragmented material:** study content exists in many formats.
2. **Poor source grounding:** a generic answer may not match the course material.
3. **Weak revision loop:** students receive explanations but not systematic practice.
4. **No mastery model:** the system often does not know which topics the learner actually understands.
5. **Low feedback quality:** students cannot easily see why an answer is wrong and which source section should be revised.

1.2 Proposed Solution

Build a personal course-aware AI tutor:

Materials → Multimodal Ingestion → Knowledge Base → RAG → Cited Tutor → Assessment → Mastery Mod

1.3 Success Criteria

A successful MVP should demonstrate:

- upload at least one PDF/PPT;
- ask a question and receive an answer grounded in the uploaded material;
- show source/page/slide references;
- generate a quiz from the material;
- grade the quiz and identify weak topics;
- generate follow-up practice based on weak topics;

- demonstrate that unsupported questions are refused or clearly marked;
- complete the end-to-end flow reliably during a live demo.

Chapter 2

2. Product Scope

2.1 MVP

Feature	MVP behavior
Authentication	Simple student account/session.
Material upload	PDF, PPTX, images; optional video support if time permits.
Parsing	Extract text plus page/slide metadata.
Knowledge base	Chunk content and store embeddings in PostgreSQL/pgvector.
Ask AI	Retrieve relevant chunks and answer using an LLM.
Citations	Show source file and page/slide for supporting evidence.
Quiz	Generate MCQ/short-answer questions from selected material.
Grading	Score answers and explain mistakes.
Mastery	Maintain topic-level correctness statistics.
Personalization	Generate revision questions for weak topics.
Dashboard	Materials, recent questions, quiz score, weak topics.

2.2 Stretch Features

- lecture-video transcription with timestamps;
- image/diagram understanding;
- semantic search over all course material;
- flashcards;
- study-plan generation;
- spaced-repetition scheduling;
- teacher/admin dashboard;
- class-level analytics;
- multilingual explanations;
- voice input/output;

- automatic topic graph.

Scope warning: Do not implement every stretch feature before the core RAG + assessment loop is stable.

Chapter 3

3. End-to-End Architecture

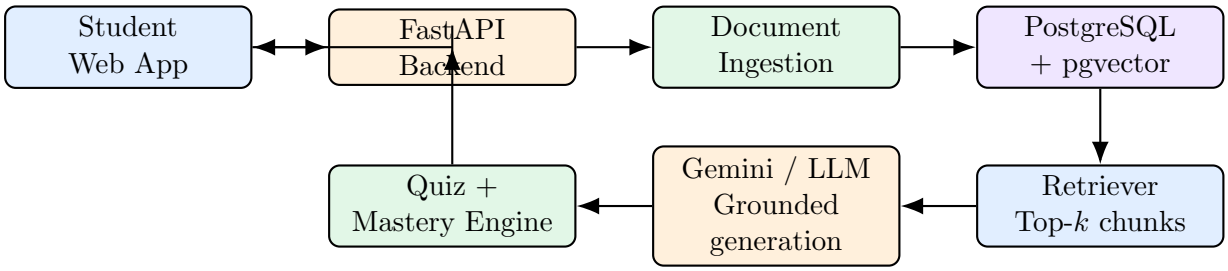


Figure 3.1: High-level system architecture

3.1 Recommended Stack

- **Frontend:** Next.js + TypeScript + Tailwind CSS.
- **Backend:** Python + FastAPI.
- **Database:** PostgreSQL.
- **Vector search:** pgvector/Supabase.
- **PDF extraction:** PyMuPDF.
- **PPT extraction:** python-pptx.
- **Video:** FFmpeg + Whisper, if enabled.
- **OCR/vision:** OCR library and/or multimodal LLM.
- **LLM:** Gemini API or another approved API.
- **Evaluation:** RAGAS/DeepEval-style checks plus a hand-built test set.
- **Deployment:** Vercel for frontend and Render for backend, with Supabase/PostgreSQL.

Chapter 4

4. Architecture by Data Flow

4.1 Upload Flow

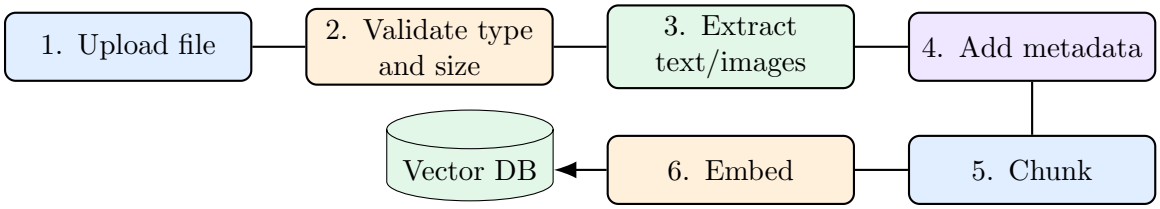


Figure 4.1: Material ingestion pipeline

For every chunk, preserve metadata such as:

- user/course identifier;
- file identifier and filename;
- page number or slide number;
- section/title;
- chunk index;
- source type;
- optional timestamp for video.

This metadata is critical for trustworthy citations.

4.2 Question Answering Flow

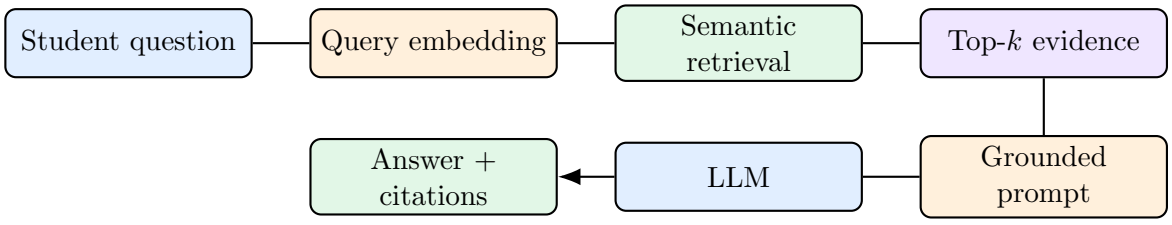


Figure 4.2: RAG question-answering flow

4.3 Grounding Rule

The model prompt should explicitly say:

> Answer using the supplied evidence. If the evidence does not contain enough information, say that the material does not provide enough evidence instead of inventing an answer. Cite the relevant source metadata.

Chapter 5

5. Retrieval-Augmented Generation (RAG)

5.1 Why RAG Instead of Training

Training or fine-tuning a large foundation model is unnecessary for the primary hackathon problem. Course documents change frequently. RAG lets the application retrieve current student material at query time.

Conceptually:

$$D = \{d_1, d_2, \ldots, d_n\}$$

where each d_i is a document chunk. An embedding function maps each chunk to a vector:

$$e_i = f(d_i).$$

For a question q:

$$e_q = f(q).$$

Retrieve the highest-similarity chunks:

$$R(q) = \text{TopK}\left(\text{sim}(e_q, e_i)\right).$$

Then construct:

$$\text{Prompt} = [\text{system rules}, q, R(q)]$$

and generate the answer from the retrieved evidence.

5.2 Chunking

Start with approximately 500–1000 tokens per chunk with modest overlap. Do not assume one chunk size works for every file. Preserve headings and page boundaries where possible.

A poor chunking strategy causes:

- missing definitions split across chunks;
- irrelevant retrieval;
- citations that point to a page containing only half the answer;
- oversized prompts.

5.3 Retrieval Improvements

If basic vector search is weak:

1. increase/decrease chunk size;
2. add metadata filters;
3. retrieve more candidates;
4. rerank candidates;
5. combine keyword and vector search;
6. remove duplicate chunks;
7. use conversation context carefully;
8. test on a fixed benchmark of real student questions.

Chapter 6

6. Multimodal Ingestion

6.1 PDF

Pipeline:

$$PDF \rightarrow \text{pages} \rightarrow \text{text blocks/images} \rightarrow \text{metadata} \rightarrow \text{chunks}.$$

For scanned PDFs, ordinary text extraction may return little or nothing. Detect this case and route the page through OCR or a vision-capable model.

6.2 PPTX

Extract:

- slide number;
- title;
- text boxes;
- speaker notes if present;
- image descriptions where possible.

6.3 Images and Diagrams

Do not blindly OCR every diagram. For diagrams, tables, flowcharts, and mathematical figures, preserve the image and use a multimodal model when interpretation is required.

6.4 Lecture Videos

Optional pipeline:

$$Video \rightarrow FFmpeg \rightarrow Audio \rightarrow Whisper \rightarrow Transcript{+}timestamps \rightarrow Chunks \rightarrow Embeddings.$$

A timestamp should be stored so the UI can point the student back to the relevant part.

Chapter 7

7. Database Design

7.1 Core Tables

A practical schema is:

Table	Important fields
users	id, name, email, created_at
courses	id, user_id, name
materials	id, course_id, filename, type, storage_url, status
chunks	id, material_id, text, embedding, page, slide, timestamp, metadata
topics	id, course_id, name
questions	id, course_id, topic_id, type, question, answer, source_ids
attempts	id, user_id, question_id, response, correct, score
mastery	user_id, topic_id, mastery, attempts, updated_at
chat_sessions	id, user_id, course_id
messages	id, session_id, role, content, citations, created_at

7.2 Mastery Model

A simple MVP model can be:

$$M_{new} = \alpha M_{old} + (1 - \alpha)S$$

where S is the score of the latest assessment and α controls how much history is retained.

For example, with $\alpha = 0.7$, a previous mastery of 0.40 and new score 0.80 gives:

$$M_{new} = 0.7(0.40) + 0.3(0.80) = 0.52.$$

This is not a scientifically complete learner model; it is an explainable hackathon MVP. Avoid presenting it as a clinically or educationally validated measurement.

Chapter 8

8. Backend API

8.1 Suggested Endpoints

Method	Endpoint	Purpose
POST	/auth/register	Create account
POST	/materials/upload	Upload material
GET	/materials	List materials
GET	/materials/id	Material status/details
DELETE	/materials/id	Delete material
POST	/chat	Ask grounded question
GET	/chat/id	Load chat history
POST	/quiz/generate	Generate quiz
POST	/quiz/submit	Grade attempt
GET	/mastery	Get topic mastery
GET	/dashboard	Dashboard summary

8.2 Backend Request Flow

```
@app.post("/chat")
async def chat(request: ChatRequest):
    query_vector = embed(request.question)
    chunks = retrieve(
        query_vector,
        course_id=request.course_id,
        top_k=6
    )
    prompt = build_grounded_prompt(
        question=request.question,
        evidence=chunks
    )
    answer = llm_generate(prompt)
    return format_answer_with_citations(answer, chunks)
```

The exact SDK calls depend on the chosen provider and should be isolated behind an LLM service interface so the application is not tightly coupled to one vendor.

Chapter 9

9. Frontend

9.1 Pages

1. Landing page.
2. Sign in / sign up.
3. Dashboard.
4. Course/material workspace.
5. Upload page.
6. AI tutor/chat.
7. Quiz page.
8. Results and explanations.
9. Mastery dashboard.
10. Settings.

9.2 Ideal Student Journey

Sign in → Create course → Upload lecture PDF → Processing → Ask question → See cited answer → Take qu

Chapter 10

10. AI Prompt Design

10.1 System Prompt Responsibilities

The system prompt should define:

- evidence-only answering;
- citation format;
- uncertainty behavior;
- concise vs detailed response modes;
- educational tone;
- quiz-generation constraints;
- prohibition on inventing source facts.

10.2 Quiz Generation Contract

Ask the model to return structured JSON, for example:

```
{
  "topic": "Binary Search",
  "questions": [
    {
      "type": "mcq",
      "question": "...",
      "options": ["A", "B", "C", "D"],
      "answer": "B",
      "explanation": "...",
      "source_ids": ["chunk_31"]
    }
  ]
}
```

Always validate the JSON on the backend. Never trust raw LLM output as database input.

Chapter 11

11. Hallucination and Safety Controls

11.1 Failure Cases

Problem	Detection	Mitigation
No relevant evidence	low similarity / empty retrieval	say insufficient evidence
Wrong citation	answer claim not supported by source	citation validation/test set
Prompt injection in document	suspicious instructions inside source	treat documents as data, not system instructions
Bad OCR	low text quality	OCR/vision fallback
Duplicate chunks	repeated retrieval	deduplication
Huge document	context overflow	chunking + top-k retrieval
API failure	timeout/status error	retry + user-visible processing state
Malformed JSON	schema validation failure	retry constrained generation

11.2 Prompt Injection Principle

A student's uploaded file is untrusted content. Instructions found inside the file must not override application/system rules.

Chapter 12

12. Evaluation

Do not judge the system only by whether the demo "looks good." Create a test set.

12.1 Test Dataset

Create at least 30–50 questions:

- direct factual questions;
- multi-step questions;
- cross-page questions;
- questions about diagrams;
- questions whose answers are absent from the material;
- ambiguous questions;
- adversarial/instruction-injection examples.

12.2 Metrics

Track:

- retrieval recall@k;
- citation correctness;
- answer groundedness;
- answer relevance;
- quiz validity;
- grading consistency;
- API latency;
- ingestion success rate.

For an internal retrieval test set:

$$Recall@k = \frac{\#\text{questions with a relevant chunk in top-k}}{\#\text{questions}}.$$

12.3 Latency Budget

Measure each component separately:

$$T_{total} = T_{upload} + T_{parse} + T_{retrieve} + T_{LLM} + T_{DB} + T_{frontend}.$$

Do not assume the LLM is always the bottleneck.

Chapter 13

13. Development Roadmap

13.1 Phase 0 — Team Alignment

Before coding, freeze:

- exact one-sentence problem;
- target student;
- MVP features;
- stack;
- repository structure;
- branch strategy;
- owner for each module.

13.2 Phase 1 — Repository

Suggested structure:

```
student-ai/
  frontend/
  backend/
  docs/
  tests/
  data/
  .env.example
  README.md
```

13.3 Phase 2 — Backend Skeleton

1. Create FastAPI app.
2. Add environment variables.
3. Add health endpoint.
4. Connect PostgreSQL.
5. Create migrations/schema.
6. Add upload endpoint.

13.4 Phase 3 — Ingestion

1. PDF extraction.
2. PPTX extraction.
3. Metadata preservation.
4. Chunking.
5. Embedding generation.
6. pgvector insertion.
7. Processing-status tracking.

13.5 Phase 4 — RAG

1. Query embedding.
2. Vector similarity search.
3. Metadata filtering.
4. Prompt construction.
5. LLM call.
6. Citation formatting.
7. Insufficient-evidence behavior.

13.6 Phase 5 — Frontend

Build only what the demo requires first:

1. upload;
2. processing state;
3. chat;
4. citations;
5. quiz;
6. results;
7. mastery dashboard.

13.7 Phase 6 — Assessment

Implement:

$$\text{Generate} \rightarrow \text{Answer} \rightarrow \text{Grade} \rightarrow \text{Update Topic Mastery} \rightarrow \text{Recommend Practice}.$$

13.8 Phase 7 — Testing

Run the fixed test set. Log every failure. Categorize failures as:

1. extraction;
2. chunking;
3. retrieval;
4. generation;
5. citation;
6. grading;
7. UI;
8. infrastructure.

13.9 Phase 8 — Deployment

1. PostgreSQL/Supabase production database.
2. Backend deployment on Render.
3. Frontend deployment on Vercel.
4. Environment variables in production.
5. CORS configuration.
6. HTTPS.
7. File-size limits.
8. Error monitoring/logging.
9. Seed demo course.

Chapter 14

14. Team Division

For a four-person team:

Person	Primary ownership	Deliverables
Member 1	AI/RAG	ingestion, chunking, embeddings, retrieval, prompts
Member 2	Backend/DB	FastAPI, PostgreSQL, auth, APIs
Member 3	Frontend	Next.js UI, chat, upload, quiz, dashboard
Member 4	Evaluation/Integration	tests, deployment, UX polish, demo data, presentation

If there are only three members, merge evaluation/integration with frontend and keep AI, backend, and frontend as the three primary owners.

Chapter 15

15. Git Workflow

Use:

```
main
develop
feature/frontend
feature/backend
feature/rag
feature/evaluation
```

Rules:

- never commit API keys;
- maintain .env.example;
- use small commits;
- pull/rebase before merging;
- test before merging;
- tag the final demo version.

Chapter 16

16. Environment Variables

Example:

DATABASE_URL=
LLM_API_KEY=
EMBEDDING_API_KEY=
SUPABASE_URL=
SUPABASE_KEY=
CORS_ORIGINS=

Never put real keys in GitHub, screenshots, frontend JavaScript, or presentation slides.

Chapter 17

17. Deployment Architecture

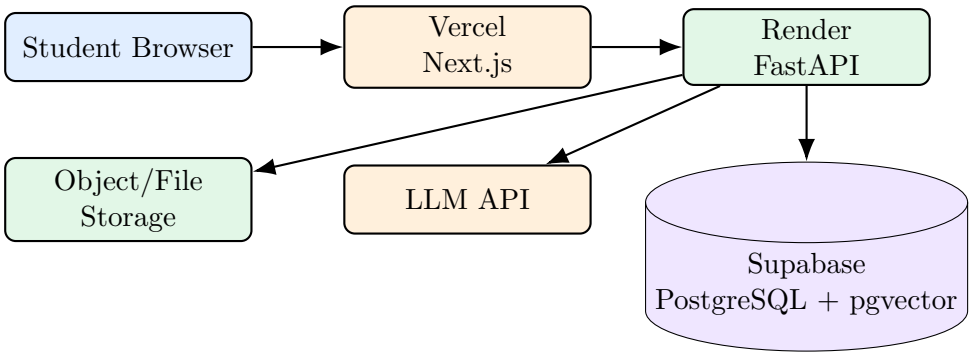


Figure 17.1: Production deployment topology

Chapter 18

18. Hackathon Demo Script

The demo should tell one story rather than showing random features.

1. Open dashboard.
2. Create “DBMS Unit 1” course.
3. Upload faculty PDF.
4. Show processing.
5. Ask a question that is answered directly from the PDF.
6. Open the citation and show the source page.
7. Ask a second, multi-step question.
8. Generate a quiz.
9. Intentionally answer one question incorrectly.
10. Show the result and weak topic.
11. Click “Practice weak topic.”
12. Show a personalized follow-up question.
13. Ask an out-of-syllabus question and demonstrate the grounded fallback.

18.1 Demo Narrative

The narrative should be:

$$\boxed{\text{My material}} \rightarrow \boxed{\text{My tutor}} \rightarrow \boxed{\text{My assessment}} \rightarrow \boxed{\text{My weak topics}} \rightarrow \boxed{\text{My next practice}}.$$

Chapter 19

19. What Can Go Wrong

19.1 API Key Failure

Symptom: 401/403 errors. Solution: verify environment variables, provider project, billing/quota, and server-side secret loading.

19.2 CORS Failure

Symptom: browser blocks backend requests. Solution: configure exact frontend origin in FastAPI CORS middleware.

19.3 Slow Upload

Cause: synchronous processing. Solution: return a processing job/status and process asynchronously where practical.

19.4 Bad Answers

Do not immediately change the LLM. First inspect:

$$\text{Extraction} \rightarrow \text{Chunking} \rightarrow \text{Retrieval} \rightarrow \text{Prompt} \rightarrow \text{Generation}.$$

The model may be receiving the wrong evidence.

19.5 Citations Look Correct but Are Wrong

A page citation alone does not prove a claim is supported. Store chunk IDs and inspect the exact retrieved text. The UI should expose enough source context to verify the claim.

19.6 Demo Internet Failure

Keep a prepared local/demo dataset and a deterministic fallback demo path. Do not fake AI results; instead, make the system visibly use pre-indexed material if live ingestion fails.

Chapter 20

20. Security and Privacy

Minimum controls:

- authentication and authorization;
- users can access only their own materials;
- file-type and file-size validation;
- malware scanning if the product moves beyond a hackathon;
- API keys only on the server;
- parameterized database queries;
- rate limiting;
- deletion endpoint;
- avoid sending unnecessary personal information to third-party APIs.

For a hackathon, clearly state what data is stored, where it is stored, and which external AI provider receives document content.

Chapter 21

21. Product Differentiation

The differentiator should not be “we use an LLM.” Many teams can call an LLM.

The defensible workflow is:

Student-owned material+source grounding+assessment+mastery tracking+adaptive practice.

This creates a closed learning loop rather than a generic chatbot.

Chapter 22

22. What Not To Do

1. Do not train GPT-2/GPT-style models just to say “we trained a model.”
2. Do not build ten UI pages before RAG works.
3. Do not support every file type on day one.
4. Do not claim mastery is scientifically accurate without validation.
5. Do not hide citations.
6. Do not rely on one manually prepared answer in the demo.
7. Do not expose API keys.
8. Do not spend hackathon time on decorative animations before reliability.
9. Do not call an answer grounded merely because the model produced a citation.

Chapter 23

23. Final Technical Checklist

Item	Done	Notes
Problem statement frozen	☐	
User journey defined	☐	
Git repository created	☐	
Frontend running	☐	
FastAPI running	☐	
Database connected	☐	
PDF parser working	☐	
PPTX parser working	☐	
Chunking implemented	☐	
Embeddings working	☐	
pgvector retrieval working	☐	
Grounded answer working	☐	
Citations working	☐	
Quiz generation working	☐	
Quiz grading working	☐	
Mastery update working	☐	
Weak-topic practice working	☐	
Evaluation dataset created	☐	
Production deployment working	☐	
Demo account/data prepared	☐	
Presentation completed	☐	
Backup demo tested	☐	

Chapter 24

24. Final System Diagram

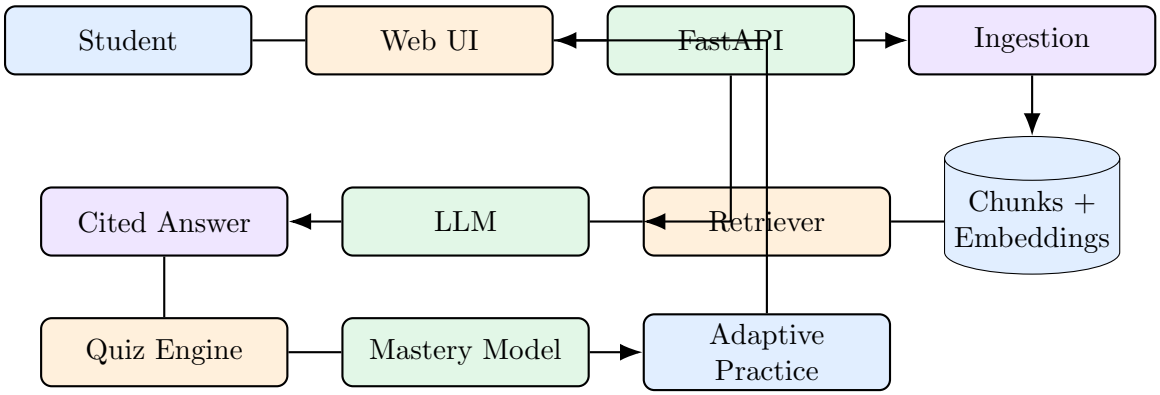


Figure 24.1: Complete student AI learning loop

One-Sentence Pitch

A source-grounded AI study companion that turns a student's own lectures and learning materials into a cited tutor, adaptive assessment system, and personalized revision loop.

Final Principle

The hackathon submission should be judged by whether the system closes the loop:

$$\boxed{\text{Material} \rightarrow \text{Understand} \rightarrow \text{Ask} \rightarrow \text{Practice} \rightarrow \text{Measure} \rightarrow \text{Identify Weakness} \rightarrow \text{Practice Again}}$$

Everything else is secondary until this loop works reliably.